Kompendium SOP 2: Synchronizacja, Pamięć, Łącza i Sieci

Szczegółowe opracowanie materiałów do kolokwium

0. Contents

1 Teoria Synchronizacji i Problemy Sprzętowe **3**
1.1 Sekcja Krytyczna (Critical Section - CS) . . . 3
1.2 Dlaczego teoria (np. Algorytm Petersona) zawodzi w praktyce? . . . 3

2 Algorytm Bankiera (Unikanie Zakleszczeń) **3**
2.1 Struktury Danych i Stany Systemu . . . 3
2.2 Zasada Działania (Weryfikacja Żądania) . . . 4
2.3 Przykładowa implementacja (weryfikacja stanu) . . . 4

3 Zaawansowana Synchronizacja POSIX **5**
3.1 Zmienne Warunkowe (Condition Variables) . . . 5
3.2 Robust Mutex (`PTHREAD_MUTEX_ROBUST`) . . . 6

4 Mechanizmy IPC: Łącza (Pipes i FIFO) **6**
4.1 Łącza anonimowe (Pipes) . . . 6
4.2 Łącza nazwane (FIFO) . . . 6

5 Pamięć Współdzielona (mmap i SHM) **7**
5.1 Pojęcie i działanie `mmap()` . . . 7
5.2 Flagi Widoczności i Ochrony Pamięci . . . 7
5.3 Dziedziczenie przy wywołaniu `fork()` . . . 7
5.4 Synchronizacja Zrzutów na Dysk (`msync`) . . . 7
5.5 Czyste obiekty SHM w systemie POSIX (`shm_open`) . . . 7

6 Głęboka Analiza Sieci Komputerowych (L2 i L3) **9**
6.1 Zasada Pakietowości i Enkapsulacja . . . 9
6.2 Warstwa L2 (Data Link - np. Ethernet) . . . 9
6.2.1 Adresacja MAC (Media Access Control) . . . 9
6.2.2 Przełącznik (Switch) i zasada jego działania . . . 9
6.2.3 Dlaczego L2 nie zbuduje Internetu? . . . 9
6.3 Warstwa L3 (Network - Internet Protocol / IP) . . . 9
6.3.1 Adresacja IP . . . 10
6.3.2 Router i Tablica Routingu . . . 10
6.4 Protokół ARP (Address Resolution Protocol) . . . 10
6.5 Anatomia skoku pakietu (Krok po kroku) . . . 10
6.6 Mechanizmy Ochronne: TTL i ICMP . . . 10

7 Warstwa Transportowa (L4) i Gniazda (Sockets) **12**
7.1 Cykl Życia Gniazd . . . 12
7.2 Protokół UDP (User Datagram Protocol) . . . 12
7.3 Protokół TCP (Transmission Control Protocol) . . . 12
7.3.1 Strumień Bajtów i Bufory . . . 12

7.3.2 Nawiązywanie połączenia (3-way handshake) 13
7.3.3 Przesyłanie danych 13
7.3.4 Zamykanie i Stany Gniazd 13
7.4 Przegląd funkcji systemowych (API Gniazd) 13

8 Pytania kontrolne do kolokwium **15**
8.1 Jak używać zmiennej warunkowej? 15
8.2 Zasymulować algorytm bankiera (Przykład) 15
8.3 Co to znaczy, że sieci są pakietowe? 15
8.4 Na jakiej podstawie router decyduje, gdzie wysłać pakiet? 16

1. Teoria Synchronizacji i Problemy Sprzętowe

1.1. Sekcja Krytyczna (Critical Section - CS)

Sekcja krytyczna to fragment kodu, w którym proces uzyskuje dostęp do współdzielonych danych. Problem synchronizacji polega na zaprojektowaniu algorytmu, który chroni ten obszar, spełniając trzy warunki:

- **Wzajemne wykluczanie:** Tylko jeden proces może przebywać w CS w danym momencie.
- **Postęp:** Jeśli CS jest pusta, decyzja o tym, kto wejdzie następny, nie może być odkładana w nieskończoność.
- **Ograniczone oczekiwanie:** Proces zgłaszający chęć wejścia do CS nie może czekać wiecznie (gwarancja braku zagłodzenia).

1.2. Dlaczego teoria (np. Algorytm Petersona) zawodzi w praktyce?

Teoretyczne algorytmy oparte wyłącznie na zmiennych współdzielonych (flagi, tury) zakładają natychmiastową i liniową widoczność zmian w pamięci. Współczesny sprzęt to niszczy:

- **Store Buffers (Efekty Cache):** Każdy rdzeń procesora ma własny bufor zapisu. Zanim zmiana flagi trafi do głównej pamięci RAM i stanie się widoczna dla innych rdzeni, procesor zdąży wykonać kolejne instrukcje.
- **Out-of-order execution (OOOE):** Jednostka sterująca procesora może dowolnie zamieniać kolejność wykonywania instrukcji, by optymalizować potok (pipeline), ignorując przy tym intencje programisty co do synchronizacji z innymi wątkami.
- **Optymalizacje kompilatora:** Kompilator zakłada środowisko jednowątkowe. Może uznać, że częste sprawdzanie flagi w pętli jest bez sensu, więc wczytuje ją raz do rejestru CPU i sprawdza w nieskończoność starą wartość.

Rozwiązaniem są dedykowane, niepodzielne instrukcje sprzętowe typu Read-Modify-Write (np. **Test-And-Set**, **Compare-And-Swap**) oraz bariery pamięciowe (Fences), które wymuszają synchronizację buforów cache.

2. Algorytm Bankiera (Unikanie Zakleszczeń)

Podstawowy protokół zapobiegania deadlockom (zakleszczeniom) w systemach, w których występuje wiele instancji tych samych zasobów (np. jednostek pamięci, urządzeń I/O). Algorytm ten sprawdza, czy przyznanie zasobów nie zamknie systemu w patowej sytuacji.

2.1. Struktury Danych i Stany Systemu

Aby algorytm zadziałał, system operacyjny musi operować na czterech wektorach/macierzach:

- **Available (Dostępne):** Ile instancji poszczególnych zasobów jest aktualnie całkowicie wolnych.
- **Max (Maksymalne):** Deklaracja największej liczby zasobów, jakiej dany proces będzie potrzebował do ukończenia pracy.
- **Allocation (Przydzielone):** Ile zasobów dany proces już aktualnie otrzymał i przetrzymuje.
- **Need (Potrzeby):** Ile zasobów proces jeszcze potrzebuje. Wzór: `Need = Max - Allocation`.

System analizując powyższe dane, operuje w jednym z dwóch stanów:

- **Stan Bezpieczny (Safe State):** Istnieje pewna sekwencja obsługi procesów (np. $P_1 \to P_3 \to P_2$), w której aktualnie *Dostępne* zasoby pozwalają dokończyć pracę pierwszemu procesowi. Ten oddaje zasoby, dzięki czemu drugi proces może dokończyć pracę, itd. Gwarantuje to brak zakleszczenia.
- **Stan Niebezpieczny (Unsafe State):** System nie potrafi znaleźć takiej sekwencji. Może to (choć nie zawsze musi) prowadzić do zakleszczenia.

2.2. Zasada Działania (Weryfikacja Żądania)

Gdy proces zgłasza *żądanie* (Request) otrzymania konkretnych zasobów, Algorytm Bankiera przechodzi przez następujące kroki:

1. Sprawdza, czy `Request` $\leq$ `Need` (czy proces nie żąda więcej niż deklarował na początku).
2. Sprawdza, czy `Request` $\leq$ `Available` (czy system w ogóle posiada tyle wolnych zasobów).
3. **Symulacja (Udawana alokacja):** System wirtualnie przyznaje zasoby. Zmniejsza w pamięci `Available`, zwiększa procesowi `Allocation` i zmniejsza jego `Need`.
4. **Algorytm Bezpieczeństwa:** System sprawdza ten nowy, wyimaginowany stan. Jeśli jest **bezpieczny** – żądanie zostaje fizycznie zrealizowane (zasoby są przyznawane). Jeśli wygenerowany stan jest **niebezpieczny** – symulacja jest cofniona, a wątek proszący o zasoby zostaje zawieszony do czasu, aż sytuacja się poprawi.

2.3. Przykładowa implementacja (weryfikacja stanu)

Poniższy kod w języku C implementuje jądro algorytmu weryfikującego, czy dany stan systemu jest bezpieczny. Scenariusz opiera się na 3 procesach i 1 typie zasobu (dokładnie jak w zadaniu kolokwialnym umieszczonym na końcu kompendium).

```c
#include <stdio.h>
#include <stdbool.h>

int main() {
    int n = 3; // Liczba procesow
    int m = 1; // Liczba typow zasobow

    // Poczatkowe struktury danych (Macierze i wektory)
    int alloc[3][1] = { {2}, {2}, {2} };
    int max[3][1]   = { {7}, {4}, {9} };
    int avail[1]    = { 3 }; // Zostalo 3 wolne instancje zasobu
    int need[3][1];

    // 1. Obliczanie macierzy Need = Max - Allocation
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }

    bool finish[3] = {false, false, false}; // Flagi zakonczenia procesow
    int safe_seq[3]; // Bufor na bezpieczna sekwencje
    int work[1];     // Zmienna robocza dla dostepnych zasobow
    for (int i = 0; i < m; i++) work[i] = avail[i];

    // 2. Szukanie bezpiecznej sekwencji
    int count = 0;
    while (count < n) {
```

```
        bool found = false;
        for (int p = 0; p < n; p++) {
            if (!finish[p]) { // Jesli proces jeszcze nie skonczyl
                int j;
                for (j = 0; j < m; j++) {
                    if (need[p][j] > work[j])
                        break; // Proces potrzebuje wiecej, niz mamy
                }

                // Jesli zasoby zaspokajaja potrzeby procesu
                if (j == m) {
                    for (int k = 0; k < m; k++) {
                        work[k] += alloc[p][k]; // Proces konczy i oddaje
                            zasoby
                    }
                    safe_seq[count++] = p; // Zapisz proces do sekwencji
                    finish[p] = true;
                    found = true;
                }
            }
        }

        // Jesli w pelnej petli zaden proces nie mogl ruszyc - Deadlock
        if (!found) {
            printf("System jest w stanie NIEBEZPIECZNYM!\n");
            return -1;
        }
    }

    // Sukces - odnaleziono sciezke bez deadlocka
    printf("Stan JEST bezpieczny.\nBezpieczna sekwencja: ");
    for (int i = 0; i < n - 1; i++) printf("P%d -> ", safe_seq[i]);
    printf("P%d\n", safe_seq[n - 1]);

    return 0;
}
```

Listing 1: Algorytm Bezpieczeństwa (Część Algorytmu Bankiera)

3. Zaawansowana Synchronizacja POSIX

3.1. Zmienne Warunkowe (Condition Variables)

Służą do oczekiwania na spełnienie konkretnego warunku logicznego. Są zawsze skojarzone z Mutexem.

```
pthread_mutex_lock(&mtx);
while (dane_nie_sa_gotowe) {
    pthread_cond_wait(&cv, &mtx);
}
// Konsumpcja danych
pthread_mutex_unlock(&mtx);
```

Listing 2: Wzorzec użycia zmiennej warunkowej

Dlaczego pętla `while` jest obowiązkowa?

1. **Kradzież sygnału (Signal Stealing):** Po otrzymaniu sygnału, wybudzony wątek wychodzi z funkcji `wait()`, która atomowo odblokowała Mutex i uspała wątek. Co kluczowe, w momencie wybudzenia – zanim funkcja `wait()` zwróci sterowanie – atomowo zamyka ona Mutex ponownie.

Mimo tego, zanim wątek zdąży zużyć zasób (np. odczytać dane), do procesora może zostać wpuszczony zupełnie inny wątek, który wejdzie do CS i "ukradnie" dane. Wybudzony wątek musi ponownie sprawdzić warunek.

2. **Wybudzenia pozorne (Spurious Wakeups):** Standard POSIX pozwala funkcji `wait()` powrócić, nawet jeśli żaden inny proces nie wywołał `signal()`, np. z powodu otrzymania systemowego przerwania (interrupt).

3.2. Robust Mutex (`PTHREAD_MUTEX_ROBUST`)

Klasyczny Mutex umieszczony w pamięci współdzielonej (używany przez procesy, nie wątki) rodzi ryzyko: "Śmierć" procesu bez zwolnienia zamka na stałe blokuje pozostałych chętnych. Z flagą `ROBUST`, po śmierci właściciela zamek nie przepada. System wybudza kolejny proces, ale zamiast 0 zwraca mu błąd `EOWNERDEAD`. Wątek ten musi "posprzątać" naruszoną strukturę danych i potwierdzić sukces funkcją `pthread_mutex_consistent()`.

4. Mechanizmy IPC: Łącza (Pipes i FIFO)

4.1. Łącza anonimowe (Pipes)

Łącza anonimowe to jednokierunkowe (half-duplex) kanały komunikacji wykorzystywane najczęściej do relacji typu producent-konsument. Cechują się tym, że nie są widoczne w hierarchii systemu plików i wymagają relacji pokrewieństwa między procesami (np. rodzic-potomek), by przekazać dostęp poprzez dziedziczenie po `fork()`.

- **Tworzenie:** Funkcja `pipe(fd)` tworzy łącze i zwraca dwa deskryptory plików: `fd[0]` otwarty do odczytu i `fd[1]` otwarty do zapisu.
- **Ograniczenia systemowe:** Łącza przechowują sekwencję bajtów bez znaczników końca i nie wspierają pozycjonowania; próba użycia `lseek` zwróci błąd `ESPIPE`. Pojemność łącza jest fizycznie ograniczona rozmiarem `PIPE_BUF`.
- **Atomowość:** Zapis bloków danych do wielkości `PIPE_BUF` jest gwarantowany jako nierozdzielny (atomic). Dla większych paczek, system może dzielić przesył na fragmenty.
- **Cykl życia i błędy:** Próba zapisu do łącza, które nie posiada aktywnego czytelnika, ustawi zmienną `errno` na `EPIPE` (broken pipe) i wyśle do procesu sygnał `SIGPIPE`. Dane pozostałe w łączu giną po zamknięciu wszystkich powiązanych deskryptorów.

4.2. Łącza nazwane (FIFO)

FIFO (Special File) to obiekt systemowy pozwalający na komunikację metodą pierwszego na wejściu, pierwszego na wyjściu, identycznie jak pipe. Zasadnicza różnica polega na tym, że FIFO posiada jawną nazwę i ścieżkę w systemie plików (np. za pomocą `mkfifo()`), dzięki czemu może być współdzielony przez procesy bez relacji pokrewieństwa.

- **Otwieranie plików:** Użycie FIFO wymaga ostrożności, ponieważ domyślnie wywołanie funkcji `open()` jest operacją blokującą. Otwarcie do odczytu (`O_RDONLY`) zablokuje proces, dopóki ktoś nie otworzy łącza do zapisu (`O_WRONLY`) i odwrotnie. Otwieranie dwukierunkowe na krzyż niesie ryzyko wystąpienia blokady systemowej (deadlock).
- **Użycie nieblokujące:** Flaga `O_NONBLOCK` zmienia standardowe działanie `open()`. Otwarcie do odczytu (`O_RDONLY | O_NONBLOCK`) kończy się natychmiastowym sukcesem, nawet jeśli nikt nie ma otwartego łącza do zapisu. Otwarcie do zapisu (`O_WRONLY | O_NONBLOCK`), gdy żaden proces nie ma otwartego łącza do odczytu, natychmiast zwraca kod -1 i ustawia `errno = ENXIO`. Co ważne, błąd i ustawienie `errno = EAGAIN` nie występuje przy funkcji `open()`, lecz pojawia się

dopiero w momencie operacji wejścia/wyjścia – wywołania `read()` (czytanie z pustego FIFO) lub `write()` (pisanie do pełnego FIFO) w trybie nieblokującym.

5. Pamięć Współdzielona (mmap i SHM)

5.1. Pojęcie i działanie `mmap()`

Tradycyjne operacje plikowe, takie jak `read()` i `write()`, wymagają kopiowania danych pomiędzy buforem jądra systemu a pamięcią użytkownika, a każdy dostęp to oddzielne wywołanie systemowe (syscall). Wywołanie `mmap()` omija te narzuty poprzez bezposrednie zmapowanie pliku (lub innego obiektu systemu operacyjnego) prosto do Wirtualnej Przestrzeni Adresowej (VMA) procesu. Zwraca to adres w pamięci, z którego proces może normalnie odczytywać i zapisywać bajty tak, jakby pracował na zwykłej tablicy.

5.2. Flagi Widoczności i Ochrony Pamięci

Przy deklaracji `mmap()`, programista określa sposób ochrony zmapowanej pamięci (np. `PROT_READ`, `PROT_WRITE`, `PROT_EXEC`) oraz kluczowe flagi zachowania:

- `MAP_SHARED`: Pozwala na mechanizm IPC (Inter-Process Communication). Zmiany w pamięci dokonane przez proces są natychmiast widoczne dla innych procesów, które zmapowały ten sam plik. Pamięć fizyczna na dysku zostaje zaktualizowana w tle.
- `MAP_PRIVATE`: Realizuje mechanizm *Copy-on-Write* (CoW). Modifikacje są stricte prywatne; zmiana wartości modyfikuje ukrytą kopię w RAM danego procesu i nie jest fizycznie zapisywana do otwartego pliku pod spodem.
- `MAP_ANONYMOUS`: Pamięć nie jest połączona z żadnym realnym plikiem i jest inicjalizowana zerami (argument deskryptora ignoruje się ustawiając go np. na -1). Mechanizm ten służy np. do czystego podziału pamięci RAM pomiędzy rodzicem a potomkiem.

5.3. Dziedziczenie przy wywołaniu `fork()`

System Linux dziedziczy wszystkie mapowania po wywołaniu `fork()` (proces potomny otrzymuje wierną kopię tablicy stron rodzica). Dla flagi `MAP_SHARED`, obaj widzą dokładnie tę samą fizyczną część pamięci. Z kolei w przypadku `MAP_PRIVATE`, załącza się mechanizm Copy-on-Write tworzący oddzielną przestrzeń.

5.4. Synchronizacja Zrzutów na Dysk (`msync`)

W przypadku `MAP_SHARED`, zapis do wirtualnej pamięci nie trafia na dysk natychmiast. Jądro jedynie flaguje zmodfikowane strony jako *DIRTY*. Procesy tła (tzw. ‘kworker‘) okresowo i asynchronicznie skanują RAM i wykonują fizyczny zrzut danych. Jeżeli aplikacja oczekuje bezwzględnej gwarancji trwałości na nośniku (np. system bazy danych), musi wywołać funkcję `msync()` z flagą `MS_SYNC`, która uśpi proces do momentu udanego zapisu na dysku.

5.5. Czyste obiekty SHM w systemie POSIX (`shm_open`)

Gdy aplikacje nie operują na fizycznym pliku, a zależy im wyłącznie na czystym komunikowaniu się przez przestrzeń pamięci dla niespokrewnionych procesów, stosuje się obiekty POSIX Shared Memory. Mają one własną, płaską przestrzeń nazw (namespace) podmontowaną zazwyczaj w katalogu podglądu debug `/dev/shm/`.

Proces użycia SHM przebiega według ściśle określonego cyklu życia:

1. **Utworzenie:** Funkcja `shm_open()` (z odpowiednimi flagami jak `O_CREAT | O_RDWR`) tworzy obiekt IPC i zwraca w postaci klasycznego deskryptora plików.

2. **Rozszerzenie:** Świeżo utworzony obszar posiada zerowy rozmiar. Należy użyć `ftruncate(fd, rozmiar)`, by zadeklarować wielkość bufora.

3. **Mappowanie i Użycie:** Użycie `mmap()` przy z flagą `MAP_SHARED` przekuwa uzyskany deskryptor w używalny adres i od tej pory sam deskryptor `fd` może zostać natychmiast zamknięty za pomocą `close()` (nie zerwie to połączenia mapowania).

4. **Likwidacja:** Obiekty SHM w przestrzeni jądra wykazują Trwałość Jądra (Kernel Persistence); będą obciążały pamięć do momentu restartu maszyny fizycznej, chyba że zniszczy się samą nazwę obiektu wywołaniem `shm_unlink()`. Sama pamięć jest jednak zwalniana finalnie dopiero gdy najpóźniejszy proces zawoła `munmap()`.

6. Głęboka Analiza Sieci Komputerowych (L2 i L3)

Sieci pełnią funkcję IPC (Inter-Process Communication) dla środowisk rozproszonych, gdzie procesy znajdują się na oddzielnych systemach operacyjnych i maszynach. Oparte są na strukturze warstwowej (stosie), z której najważniejsze to warstwa Łącza Danych (L2) i Warstwa Sieciowa (L3).

6.1. Zasada Pakietowości i Enkapsulacja

Internet nie przesyła ciągłych, bezkształtnych strumieni danych. Transmisja jest podzielona na małe porcje nazywane **pakietami**. Każdy pakiet składa się z:

- **Payload (Ładunek):** Właściwe dane generowane przez aplikację użytkownika.
- **Headers (Nagłówki):** Metadane dodawane przez każdą warstwę.

Enkapsulacja (Hermetyzacja): Dane z warstwy wyższej są traktowane jako sam ładunek w warstwie niższej. Aplikacja generuje dane → L4 (TCP/UDP) dokleja numery portów → L3 (IP) dokleja adresy routerów → L2 (Ethernet) dokleja sprzętowe adresy MAC.

6.2. Warstwa L2 (Data Link - np. Ethernet)

Warstwa działająca wyłącznie w pojedynczym, ograniczonym fizycznie segmencie sieci (w sieci lokalnej, LAN).

6.2.1. Adresacja MAC (Media Access Control)

Adresy MAC to 48-bitowe ciągi znaków "wypalone" na kartach sieciowych przez producentów. Najważniejsza cecha MAC: to adres **płaski (flat)**. Nie ma struktury geograficznej. Po MACu nie stwierdzisz, w jakim kraju jest dane urządzenie (to tak jak numer seryjny telefonu).

6.2.2. Przełącznik (Switch) i zasada jego działania

Urządzeniem łączącym urządzenia w L2 jest Switch. Przełącznik podejmuje decyzję o przekazaniu tzw. "ramki" (frame) wyłącznie na podstawie adresów MAC. Switch analizuje każdą przychodzącą ramkę i realizuje dwie równoległe operacje:

1. **MAC Learning (Uczenie się):** Switch czyta *źródłowy* adres MAC ramki i zapisuje go w swojej wewnętrznej tablicy (MAC Table), wiążąc ten adres z fizycznym portem (kablem), z którego przyszła.
2. **Forwarding (Przekazywanie):** Switch czyta *docelowy* adres MAC i szuka go w tablicy.
   - Jeśli go znajdzie, wysyła ramkę **tylko** na ten konkretny port (unicast).
   - Jeśli nie zna tego adresu, wykonuje **Flooding (Zalewanie)** – wysyła kopię ramki na *wszystkie* porty (poza tym, z którego przyszła), licząc, że właściciel sam rozpozna pakiet i na niego odpowie.

6.2.3. Dlaczego L2 nie zbuduje Internetu?

Mechanizm Floodingu działa w biurze (10 komputerów), ale wyklucza globalne skalowanie. Gdyby połączyć miliony komputerów jednym, globalnym Switchem, zapytanie o jeden nieznany adres w Sydney spowodowałoby rozesłanie kopii do każdego telefonu i PCta na planecie, natychmiastowo paraliżując łącza.

6.3. Warstwa L3 (Network - Internet Protocol / IP)

L3 naprawia problemy L2. Wprowadza pojęcie routingu (trasowania), dzieląc światowy internet na segmenty (podsieci).

6.3.1. Adresacja IP

Adresy IP to numery **hierarchiczne**. Z punktu widzenia routingu adres IP dzieli się na *Network ID* (Adres sieci / Kod pocztowy) oraz *Host ID* (Adres maszyny wewnątrz podsieci). Pozwala to routerom na grupowanie decyzji (np. "Wszystko, co zaczyna się na 193.x.x.x ślij w lewo").

6.3.2. Router i Tablica Routingu

Router to "strażnik graniczny" L3. Każdy router posiada **Tablicę Routingu (Routing Table)**, która zawiera wpisy mówiące: "Dla danej sieci docelowej, użyj tego portu, by przesłać pakiet do tego konkretnego urządzenia dalej (Next Hop)".

Weryfikacja pakietu odbywa się za pomocą **Longest Prefix Match (Najdłuższego dopasowania prefiksu)**: Jeśli router otrzymuje pakiet na adres docelowy `192.168.1.5`, sprawdza wpisy w tablicy. Może mieć trasę dla szerokiej sieci `192.0.0.0` (Europa) oraz trasę dla `192.168.1.0` (Sala 112). Router wybierze Salę 112, bo to dopasowanie jest dłuższe (dokładniejsze). Jeśli adres kompletnie nie pasuje do żadnej reguły, Router wysyła pakiet do **Domyślnej Bramy (Default Gateway)**. Co ważne: **Router nigdy nie stosuje zalewania (floodingu)**.

6.4. Protokół ARP (Address Resolution Protocol)

Warstwy L3 i L2 działają razem, ale posługują się innym nazewnictwem (IP vs MAC). Do płynnego łączenia tych światów służy ARP. Jeśli komputer chce wysłać plik na adres `192.168.1.50` w tej samej lokalnej sieci, nie może tego zrobić, dopóki nie zdobędzie jego MACa. Wysyła więc przez Ethernet pakiet specjalny **ARP Request** (wypuszczony na adres MAC typu Broadcast: FF:FF:FF:FF:FF:FF), "Krzycząc" do sieci L2: *"Kto z Was zarządza adresem IP 192.168.1.50? Niech poda swój adres MAC!"*. Zgłoszona odpowiedź ładuje się do pamięci komputera.

6.5. Anatomia skoku pakietu (Krok po kroku)

Co fizycznie dzieje się z adresami, gdy pakiet podróżuje z Twojego Laptopa (PC1) przez Router (R1) do Serwera(S1)?

1. **Generowanie (PC1):** Pakiet IP powstaje w OSie. Adres IP(Źródło) to PC1, Adres IP(Cel) to S1.
2. **Decyzja (PC1):** OS dostrzega, że S1 nie ma w jego sieci. Używa ARP do zdobycia adresu MAC Routera (R1).
3. **Zejście do L2 (PC1 → R1):** Powstaje ramka. MAC(Źródło): PC1, MAC(Cel): R1. Pakiet rusza po kablu do Routera.
4. **Przetwarzanie w Routerze (R1):** Router ściąga stary nagłówek MAC. Patrzy na niezmieniony adres IP(Cel) = S1. Przeszukuje Tablicę Routingu. Ustala, którym kablem wysłać pakiet dalej i jaki jest adres MAC serwera S1.
5. **Ponowna Enkapsulacja (R1 → S1):** Router buduje NOWY nagłówek L2! MAC(Źródło) to teraz MAC Routera R1, a MAC(Cel) to S1. Wyrzuca go kablem do serwera.

Konkluzja: Adresy IP wędrują niezmienione (End-to-End), natomiast adresy MAC zmieniają się w całości po każdym przeskoczeniu przez router (Hop-to-Hop).

6.6. Mechanizmy Ochronne: TTL i ICMP

- **TTL (Time To Live):** Specjalne pole w nagłówku IPv4 (początkowo np. 64 lub 128). Każdy router, przez który przechodzi pakiet, obniża TTL o 1. Jeśli z powodu pętli routingu (błędy w sieci) pakiet wędruje w kółko, router, który zmniejszy TTL do wartości 0, musi **bezwzględnie zniszczyć** (porzucić) ten pakiet. Gwarantuje to samooczyszczanie się Internetu.

- **ICMP (Internet Control Message Protocol):** Narzędzie asystujące dla IP. Służy do raportowania incydentów. Gdy router zabija pakiet z powodu TTL=0, generuje i odsyła do oryginalnego nadawcy komunikat ICMP *Time Exceeded*. Z usług ICMP wprost korzystają takie programy jak `ping` (wysyła ICMP Echo Request).

7. Warstwa Transportowa (L4) i Gniazda (Sockets)

Zadaniem Warstwy Transportowej (L4) jest realizowanie połączeń proces-proces, bazując na adresacji urządzeń (host-to-host) dostarczanej przez protokół IP w warstwie L3. System operacyjny wie, do jakiej aplikacji dostarczyć otrzymane bajty, przypisując każdemu połączeniu abstrakcyjny obiekt zwany gniazdem (socket) powiązany z unikalnym numerem portu. Globalnie unikalny adres punktu końcowego aplikacji w sieci definiuje się jako parę [adres IP]:[port].

7.1. Cykl Życia Gniazd

Aplikacja komunikująca się w standardzie POSIX musi utworzyć w jądrze odpowiedni obiekt za pomocą systemowego wywołania `socket(domain, type, protocol)`. Nowo utworzone gniazdo istnieje w jądrze, lecz jest "puste" – nie posiada powiązanego z nim adresu.

- **Adresacja:** Aby odebrać pakiety wysłane na konkretny port serwera, należy wywołać funkcję `bind()`, która łączy gniazdo z lokalnym portem i adresem IP. Aplikacje pełniące rolę klienta zwykle omijają wywołanie tej funkcji – system operacyjny przy pierwszym wysłanym pakiecie automatycznie i losowo dobierze z puli wolny port (ephemeral port).
- **Porządek Bajtów (NBO):** Różne architektury maszyn mogą różnie zapisywać typy wielobajtowe w pamięci. Interfejs gniazd wymaga, aby dane sterujące sieci (np. porty i adresy IP) były przekazywane z użyciem Sieciowego Porządku Bajtów (Network Byte Order, NBO), który zawsze jest tożsamy ze standardem Big Endian. Do konwersji używa się wbudowanych funkcji z rodziny `inet_pton()` lub `inet_ntop()`.

7.2. Protokół UDP (User Datagram Protocol)

Najprostszy protokół L4. Charakteryzuje się brakiem pojęcia połączenia (brak uścisku dłoni czy procedury rozłączania) co czyni go w dużej mierze bezstanowym dla jądra. Przesyła on dyskretne, niepodzielne wiadomości znane jako Datagramy.

- Posiada zerowe gwarancje niezawodności. Pakiety UDP mogą ulec po cichu zagubieniu, mogą zostać omyłkowo zduplikowane lub dotrzeć do odbiorcy w innej kolejności, niż zostały nadane.
- Protokół natywnie wspiera mechanizmy rozgłaszania typu Multicast i Broadcast (co w przypadku TCP jest niemożliwe).
- Pojedyncze gniazdo UDP potrafi równocześnie odbierać i obsługiwać datagramy z różnych i niezależnych źródeł, przy użyciu wywołań `sendto()` i `recvfrom()`.

7.3. Protokół TCP (Transmission Control Protocol)

Złożony i wymagający wcześniejszego zestawienia połączenia (connection-oriented) protokół, gwarantujący w pełni niezawodny, ułożony we właściwej kolejności, dwukierunkowy przepływ danych.

7.3.1. Strumień Bajtów i Bufory

Urządzenia rozmawiające po TCP nie wymieniają pojedynczych wiadomości, ale transmitują ciągły, abstrakcyjny Strumień Bajtów. Ten wirtualny strumień jest przez system na bieżąco krojony na mniejsze części zwane segmentami. Do poprawnego śledzenia pozycji danego segmentu, TCP używa wbudowanych 4-bajtowych Liczników Sekwencyjnych (Sequence Number), do których odbiorca z powrotem odsyła Numery Potwierdzeń (Acknowledgement Number), powiadamiające nadawcę o udanym odebraniu danych i ewentualnej prośbie o retransmisję przy wystąpieniu utraty danych. Otrzymywane i wysyłane segmenty są składowane w jądrze OS odpowiednio w buforze wejściowym (Rx) oraz wyjściowym (Tx).

7.3.2. Nawiązywanie połączenia (3-way handshake)

Para połączonych gniazd określana jest poprzez precyzyjną 4-krotkę: `[Adres IP klienta, Port klienta, Adres IP serwera, Port serwera]`. Proces uzyskiwania połączenia zaczyna się od klasycznego uścisku dłoni (3-way handshake). 1. Klient inicjalizujący wymianę używa flagi `SYN`. 2. Serwer odpowiada flagami `SYN+ACK`, a klient ostatecznie potwierdza to flagą `ACK`.

Po stronie klienta odpowiada za to wywołanie blokującej funkcji `connect()`. Z kolei serwer rezerwuje specjalne Gniazdo Nasłuchujące (Listening Socket), które trzyma kolejkę przychodzących żądań (utworzoną przez `listen()`). Wówczas wywołanie funkcji `accept()` ściąga gotowe żądanie z kolejki (backlog) i wypluwa zupełnie nowy deskryptor do dedykowanego Gniazda Klienta. Od tego momentu po stronie serwera istnieje $n + 1$ gniazd: jedeno serwerowe i n gniazd połączonych z poszczególnymi stacjami zdalnymi.

7.3.3. Przesyłanie danych

Ze względu na to, że TCP jest zorientowane strumieniowo, dane zapisane za pomocą pojedynczego wywołania `write()` (lub `send()`) mogą dotrzeć do klienta w paczkach, co zmusza go do kilkukrotnego wywołania funkcji odbierającej (`read()` / `recv()`). Należy więc zawsze umieszczać funkcje czytające/piszące we wnętrzu pętli zabezpieczających (wystąpić może również przypadek, kiedy bufor jądra jest zapchany, co sprawi, że funkcja przyjmie/zwróci bajtów mniej niż zakładał limit).

7.3.4. Zamykanie i Stany Gniazd

Proces rozłączenia następuje dwustronnie (niezależnie), za pomocą wysłania flagi `FIN` (sygnalizuje, że stacja od teraz nie wyśle już do strumienia nic, co będzie wymagało potwierdzenia).

- **Rozłączenie prawidłowe (Graceful):** Typowo polega na użyciu funkcji `close()`. Otrzymanie na gniazdku cudzego pakietu `FIN` objawi się tym, że próba czytania `recv()` nie zostanie zablokowana lecz bezpiecznie zwróci kod 0. Mimo tego sam `send()` na gniazdo dalej odniesie sukces – zdalne przesłanie `FIN` wcale nie oznacza bowiem, że druga strona jest w pełni martwa.

- **Rozłączenie nagłe (Abortive):** Jeżeli dojdzie do sytuacji wymuszonego lub wadliwego rozłączenia na gniazdo wysyłana jest natychmiastowa flaga uśmiercająca `RST`. Próby komunikacji po napotkaniu `RST` zwrócą w `errno` błąd `ECONNRESET`, a wywołanie funkcji `send()` wyrzuci również na poziomie systemu przerwanie sprzętowe `SIGPIPE`.

- **Stan TIME_WAIT:** Strona połączenia, która była pierwszą zgłaszającą chęć uśmiercenia sesji (ta, która wysłała w internet pakiet `FIN` jako pierwsza), po ukończeniu cyklu wymiany potwierdzeń wchodzi w bezwarunkowy stan oczekiwania (tzw. `TIME_WAIT`) trwający $2 \times MSL$ (Maximum Segment Lifetime), co na Linuksie jest sztywno zdefiniowane na wartość 60 sekund. Zabezpiecza to gniazdo i całą sieć przed sytuacją, w której to nagle omyłkowo ukatrupiony pakiet potwierdzający (`ACK`) wędrujący z dużą czkawką w sieci uderza rykoszetem w nowy proces, który zdążył zaalokować i "zarezerwować" port po zmarłym poprzedniku na tym samym komputerze. By zapobiec rzucaniu błędu przez funkcję `bind()`, używa się popularnej flagi `SO_REUSEADDR` przed samym wykonaniem bindowania serwera.

7.4. Przegląd funkcji systemowych (API Gniazd)

Poniżej zestawiono kluczowe funkcje systemowe (syscalls) wykorzystywane do obsługi gniazd sieciowych w standardzie POSIX, zaprezentowane na wykładzie:

- `socket(domain, type, protocol)` – Tworzy nowy punkt końcowy komunikacji (gniazdo) w jądrze systemu i zwraca jego deskryptor (`fd`). Argument `domain` określa rodzinę adresów (np. `AF_INET` dla IPv4), `type` określa typ komunikacji (np. `SOCK_DGRAM` dla UDP, `SOCK_STREAM` dla TCP), a `protocol` zazwyczaj wynosi 0 (domyślny protokół).

- `bind(sockfd, addr, addrlen)` – Przypisuje lokalny adres IP oraz port (przekazane w strukturze np. `sockaddr_in`) do utworzonego, "pustego" gniazda. Typowo wywoływane przez serwery.
- `inet_pton()` / `inet_ntop()` – Funkcje pomocnicze dokonujące konwersji adresów IP pomiędzy czytelną formą tekstową (np. "192.168.0.1"), a postacią binarną w sieciowym porządku bajtów (NBO).
- `getsockname()` / `getpeername()` – Zwracają przypisany do gniazda adres lokalny (`getsockname`) lub adres podłączonego, zdalnego hosta (`getpeername`).
- `sendto()` / `recvfrom()` – Podstawowe funkcje do wysyłania i odbierania pojedynczych datagramów w protokole UDP. Wymagają (lub zwracają) każdorazowego podania adresu docelowego/źródłowego dla konkretnego datagramu.
- `listen(sockfd, backlog)` – Przełącza gniazdo TCP serwera w tryb pasywnego nasłuchiwania na nadchodzące połączenia (rozpoczyna obsługę uścisków dłoni) i definiuje maksymalny rozmiar kolejki (backlog) dla połączeń oczekujących na akceptację.
- `accept()` – Pobiera pierwsze w pełni nawiązane połączenie z kolejki gniazda nasłuchującego i zwraca całkowicie nowy deskryptor gniazda dedykowany do komunikacji wyłącznie z tym jednym klientem.
- `connect(sockfd, addr, addrlen)` – Używane przez klienta TCP do aktywnego nawiązania połączenia (inicjuje 3-way handshake). Zazwyczaj blokuje wątek do momentu uzyskania odpowiedzi od serwera (segment `SYN+ACK`).
- `send()` / `recv()` – Służą do zapisu i odczytu strumienia bajtów w nawiązanym już połączeniu TCP. Zwracają liczbę pomyślnie przetworzonych bajtów (która może być mniejsza niż zażądana, co wymusza użycie pętli).
- `close(fd)` – Informuje jądro systemu, że proces nie wyśle już więcej danych ani nie jest zainteresowany odbiorem kolejnych. Zleca w tle asynchroniczne wysłanie segmentu `FIN` i ostateczne zwolnienie zasobów (zamknięcie deskryptora).
- `shutdown(fd, how)` – Pozwala na częściowe zamknięcie połączenia. Przykładowo `shutdown(fd, SHUT_WR)` zamyka jedynie strumień wyjściowy (wysyła `FIN`), ale pozostawia możliwość dalszego odbierania danych z bufora.

8. Pytania kontrolne do kolokwium

8.1. Jak używać zmiennej warunkowej?

Zmienna warunkowa (Condition Variable, np. `pthread_cond_t`) służy do usypiania wątku do momentu spełnienia konkretnego logicznego warunku. Należy trzymać się sztywnego wzorca:

- Zawsze używaj jej w parze z Mutexem. Zablokuj Mutex przed sprawdzeniem warunku.
- Sprawdzaj warunek w pętli `while` (nigdy `if`), aby uchronić się przed kradzieżą sygnału (zanim wątek fizycznie się obudzi i zamknie zamek, inny wątek zdąży zużyć zasób) oraz przed tzw. wybudzeniami pozornymi (Spurious Wakeups).
- Wywołaj funkcję `wait()` (która atomowo odblokowuje Mutex i usypia wątek, a w momencie wybudzenia – zanim powróci z funkcji – atomowo zamyka Mutex ponownie).
- Po wykonaniu pracy (konsumpcji zasobu) odblokuj Mutex i (opcjonalnie) powiadom inne wątki za pomocą `signal()` lub `broadcast()`.

8.2. Zasymulować algorytm bankiera (Przykład)

Zakładamy system z jednym typem zasobu i 3 procesami. Łącznie mamy 9 instancji zasobu.

- **Obecny Stan (Allocation):** P_0 trzyma **2**, P_1 trzyma **2**, P_2 trzyma **2** (Razem w użyciu: 6)
- **Zadeklarowany Max:** P_0 do końca chce **7**, P_1 chce **4**, P_2 chce **9**
- **Potrzeby (Need = Max - Alloc):** P_0 brakuje **5**, P_1 brakuje **2**, P_2 brakuje **7**
- **Wolne (Available):** $9 - 6 =$ **3 instancje**

Symulacja szukania bezpiecznej sekwencji:

1. Szukamy procesu, którego `Need` $\leq$ `Available` (3). Pasuje tylko P_1 (`Need` $= 2$).
2. Zaspokajamy P_1. P_1 kończy pracę i oddaje swoje zasoby (2 z powrotem do puli). Nowe `Available` to $3 + 2 =$ **5**.
3. Szukamy dalej (`Need` ≤ 5). Pasuje P_0 (`Need` $= 5$).
4. Zaspokajamy P_0. P_0 kończy pracę i oddaje swoje zasoby (2 z powrotem do puli). Nowe `Available` to $5 + 2 =$ **7**.
5. Szukamy dalej (`Need` ≤ 7). Pasuje P_2 (`Need` $= 7$).
6. Zaspokajamy P_2. P_2 kończy pracę i oddaje zasoby. Nowe `Available` $=$ **9**.

Wniosek: Stan jest **bezpieczny**, ciąg zdarzeń $\langle P_1, P_0, P_2 \rangle$ pozwala na bezproblemowe ukończenie pracy systemu bez zakleszczenia.

8.3. Co to znaczy, że sieci są pakietowe?

Oznacza to, że dane nadawane przez aplikacje nie płyną przez medium transmisyjne (kable, fale) jako jeden nieprzerwany, monolityczny strumień. Zamiast tego są dzielone na mniejsze, niezależne porcje nazwane **pakietami** (lub ramkami/datagramami zależnie od warstwy). Każdy z takich pakietów otrzymuje własny nagłówek z adresem nadawcy i odbiorcy. Dzięki temu wieloma pakietami z różnych aplikacji (lub od różnych użytkowników) można w pełni zarządzać niezależnie – mogą podróżować różnymi trasami i "przeplatać się" we wspólnym medium transmisyjnym.

8.4. Na jakiej podstawie router decyduje, gdzie wysłać pakiet?

Router operuje w warstwie sieciowej (L3). Odbierając pakiet:

- Odczytuje docelowy adres IP z nagłówka.
- Zagląda do swojej Tablicy Routingu (Routing Table).
- Szuka rekordu dopasowując docelowy adres z nagłówka do adresów sieci w tablicy, wykorzystując zasadę **Longest Prefix Match (Najdłuższego dopasowania prefiksu)** – wybiera tę trasę, która jest "najwęższa" (czyli w bitach pasuje najdłużej).
- Gdy znajdzie idealne dopasowanie, kieruje pakiet na konkretny port wyjściowy (interfejs) i przesyła go do urządzenia określonego jako *Next Hop* (Następny Skok). Jeśli nie znajdzie dopasowania, kieruje pakiet do Domyślnej Bramy.